We must also perform the following actions before shrinking an index:

- *The source index must be switched off (made read-only) for indexing.* Although not mandatory, it is recommended that we also turn off the replicas before shrinking.
- *The target index must not be created or exist before the shrinking action.*
- *All index shards must reside on the same node.* We must set the `index.routing.allocation.require.<node_name>` property on the index with the node name to achieve this.
- *The target index's shard count must be a factor of the source index's shard number.* Our `all_cars` index with 50 shards can only be shrunk to 25, 10, 5, or 2 shards.
- *The target index's node must satisfy the memory requirements.*

We can use shrinking operations when we have many shards but the data is sparsely distributed. As the name suggests, the idea is to reduce the number of shards.

Splitting or shrinking indexes is a nice way to manage them as our data grows. Another advanced operation is to create indexes on a set pattern with the help of a rollover mechanism.

6.8.3 Rolling over an index alias

Indexes accumulate data over time. Yes, we can split an index to handle additional data, as we saw earlier. However, splitting simply readjusts the data into additional shards. Elasticsearch provides another mechanism called *rollover*, where the current index is automatically rolled over to a new blank index.

Unlike in a splitting operation, in a rollover, documents are not copied to the new index. The old index becomes read-only, and any new documents are indexed into this rolled-over index from now on. For example, if we have an index `app-000001`, rolling over creates a new index `app-000002`. If we roll over once again, another new index, `app-000003`, is instantiated, and so on.

The rollover operation is frequently used when dealing with time-series data. Time-series data—data generated for a specific period like every day, week, or month—is usually held in an index created for a particular period. Application logs, for example, are created based on the date: `logs-18-sept-21`, `logs-19-sept-21`, and so on.

This will be easy to understand when you see it in action. Let's say we have an index for cars: `cars_2021-000001`. Elasticsearch performs a few steps to roll over this index:

1. Elasticsearch creates an alias pointing to the index (in this case, `cars_2021-000001`). Before it creates the alias, we must ensure that the index is writable by setting `is_write_index` to `true`. The idea is that the alias must have at least one writable backing index.
2. Elasticsearch invokes a rollover command on the alias using the `_rollover` API. This creates a new rollover index (for example, `cars_2021-000002`).

NOTE The trailing suffix (such as `000001`) is a positive number, and Elasticsearch expects the index to be created with it. Elasticsearch can only increment from a positive number; it doesn't matter what the starting number is.